MRISB 4-STEP LESSON DESIGN | UbD × ETEC

OUTCOME → PROOF → PREPARE → RESPOND → REFLECT *Plan the learning first; let the checklist follow. Click a field to type · click a box to tick it.*

Teacher	Subject	Grade / Section	Date / Period	Unit / Topic
Mr. Ashraf Elbialy	Computer Science (IGCSE 0478)	Grade 10 - Classes 10BR1 to 10BR3	Wk 8 (11-15 Oct 2026) - Period 1	Ch 8 - Programming task 8.1 (input and output)

Pacing (2-1-1-2): ☑ On plan ☐ Ahead ☐ Behind

Weeks ahead / behind: 0 **Week / lesson:** Week 8 / Lesson 1 of 2

Starting point (2-2-1-3): *What did my last check (quiz, exit ticket, homework) show — and what am I changing today because of it?*

Data dictionary homework: 75% correct types; casting still shaky. Today: apply types in a full input-process-output program (Programming task 8.1).

1 OUTCOMES — What must students learn?

Desired learning outcome(s) — *trace to the curriculum / standard and the table of specifications (ToS)*

Students use INPUT and OUTPUT with prompts and clear messages to complete Programming task 8.1, applying variables, constants and data types. Syllabus 8.1.1-8.1.3; AO2, AO3.

Success criteria — *what will successful learning look / sound like?*

My program asks clear prompts, converts inputs to the right type, processes them and outputs a readable message; my pseudocode matches my Python code.

Key knowledge / skill / understanding:

Input-process-output model; OUTPUT can join text and variables; prompts guide the user.

Likely misconception / access or challenge need:

OUTPUT and INPUT must be on one line; print adds spaces automatically everywhere.

2 PROOF — What will students do to show they have learned it?

Three activities, each with a DIFFERENT strategy, resource and evidence tool. Then tick the Coverage Check on page 2.

ACTIVITY 1 *student task + evidence of learning*

Strategy: Discussion / dialogu **Time:** 6 min

What students will do:

Look at two versions of the same program (one with no prompts/messages, one user-friendly). Discuss what makes output clear for any user, including EAL users.

Evidence tool: Oral questioning

Assessment: ☑ Diagnostic ☐ Formative ☐ End

Grouping: ☐ Individual ☐ Pair ☑ Group

Resources:

Two program demos.

Support (low performers / SEN):

Picture of input-process-output diagram.

Stretch (fast learners / gifted):

Suggest an accessibility improvement.

ACTIVITY 2 *student task + evidence of learning*

Strategy: Project-based task **Time:** 24 min

What students will do:

Programming task 8.1: plan with an IPO table, write pseudocode, then code in Python a 'travel cost calculator' (distance, fuel price, passengers -> cost per person). Test with 3 sets of data.

Evidence tool: Practical / performance task

Assessment: ☐ Diagnostic ☑ Formative ☐ End

Grouping: ☑ Individual ☐ Pair ☐ Group

Resources:

Coursebook (CUP 9781108915144) Programming task 8.1; IPO table template; test table.

Support (low performers / SEN):

IPO table pre-filled for inputs; code skeleton.

Stretch (fast learners / gifted):

Add currency choice (SAR/USD) using a constant exchange rate.

ACTIVITY 3 *student task + evidence of learning*

Strategy: Cooperative learning **Time:** 7 min

What students will do:

Pair test swap: run partner's program with normal, boundary and silly data; record results in a test table and give one improvement.

Evidence tool: Peer-assessment checklist

Assessment: ☐ Diagnostic ☑ Formative ☐ End

Grouping: ☐ Individual ☑ Pair ☐ Group

Resources:

Test table template.

Support (low performers / SEN):

Test data given.

Stretch (fast learners / gifted):

Suggest validation to stop silly data.

3 PREPARE — What do I need to do so students can succeed?

Set students up to succeed:

☑ Hook / prior learning ☑ Explain or model ☑ Success criteria shared

☑ Key vocabulary / examples ☑ Real-life connection ☑ Clear instructions / resources

☑ Praise, recognition and reinforcement planned

My setup / modelling: Time: 6 min

Hook: a ticket machine that says nothing - how do you use it? Model IPO table, then pseudocode: INPUT Distance; OUTPUT "Cost per person: ", Cost. Translate to Python with float(input('...')) and print(). Share success criteria.

What is new or innovative in this lesson? *(ETEC looks for this)*

Students plan with an IPO table and test with three kinds of data, following the full design-code-test cycle used in Paper 2 scenario questions.

Progress recorded / shared:

☑ Gradebook ☐ Parents ☑ Written feedback in notebook / portfolio

4 RESPOND — What will I do with what I see and hear?

If students struggle or finish early I will:

☑ Re-teach ☑ Regroup ☑ Give extra support

☑ Stretch ☑ Have students redo / improve

What I will watch for / likely responses:

No type conversion; prompts missing; OUTPUT joined with + causing TypeError; division by zero passengers. Likely: 'it works with my numbers' - push for test data.

Close / exit check *(summative + time for learners' questions)* Time: 7 min

Exit ticket: write pseudocode lines for input of Name and output of a greeting; name one test you ran. Learners' questions.

Homework / platform *(e.g. Madrasati)*

Finish task 8.1 and upload code + test table. Read coursebook 8.4 arithmetic operators.

5 REFLECT — After the lesson

Complete within a day. This is reflective-practice evidence and feeds your next Starting point.

What worked — and how do I know?

Travel context worked well - students used routes to their home cities. Peer testing found real bugs (zero passengers). 80% completed with test table.

What will I change next time?

Model the test table with one example row first; several students tested only 'normal' data.

Students needing follow-up:

10BR2: 4 students did not finish coding; extra lab time offered. Gifted students: currency extension done by 5.

COVERAGE CHECK

Tick where each must-have happens (1 · 2 · 3 = activity, C = close, H = homework). An empty box is a gap to fix before teaching. Literacy and numeracy apply in every subject.

Literacy read + write · 2-1-1-6	Numeracy numbers / data · 2-1-1-6	Digital students use it · 2-1-1-4	Digital research trusted sources · 2-1-1-4	Real-life ① application · 2-1-1-5	Real-life ② application · 2-1-1-5	Inquiry experiment / research · 2-1-1-7	Thinking problem-solving · 2-1-1-7
☑1 ☐2 ☐3 ☑C ☐H	☐1 ☑2 ☐3 ☐C ☐H	☐1 ☑2 ☑3 ☐C ☑H	☐1 ☐2 ☐3 ☐C ☑H	☐1 ☑2 ☐3 ☐C ☐H	☑1 ☐2 ☐3 ☐C ☐H	☐1 ☐2 ☑3 ☐C ☐H	☐1 ☑2 ☐3 ☐C ☐H
Create something new · 2-1-1-7	**Discussion** expressing ideas · 2-1-1-8	**Pair / group** cooperative work · 2-1-1-8	**Learner choice** independent work · 3-2-1-6	**Self-assessment** 2-2-1-1	**Peer assessment** 2-2-1-1	**SEN / low** specific support · 2-1-1-3	**Gifted** specific stretch · 2-1-1-3
☐1 ☑2 ☐3 ☐C ☐H	☑1 ☐2 ☐3 ☐C ☐H	☑1 ☐2 ☑3 ☐C ☐H	☐1 ☑2 ☐3 ☐C ☐H	☐1 ☐2 ☐3 ☑C ☐H	☐1 ☐2 ☑3 ☐C ☐H	☑1 ☑2 ☑3 ☐C ☐H	☑1 ☑2 ☑3 ☐C ☐H

ALWAYS VISIBLE IN DELIVERY: Safe, respectful climate • every learner drawn in • purposeful use of time • smooth transitions • learner voice • specific feedback and next steps • students act on feedback

MRISB 4-STEP LESSON DESIGN | UbD × ETEC

OUTCOME → PROOF → PREPARE → RESPOND → REFLECT *Plan the learning first; let the checklist follow. Click a field to type · click a box to tick it.*

Teacher	Subject	Grade / Section	Date / Period	Unit / Topic
Mr. Ashraf Elbialy	Computer Science (IGCSE 0478)	Grade 10 - Classes 10BR1 to 10BR3	Wk 8 (11-15 Oct 2026) - Period 2	Ch 8 - 8.4 Arithmetic operators and 8.5 Sequence

Pacing (2-1-1-2): ☑ On plan ☐ Ahead ☐ Behind

Weeks ahead / behind: 0 **Week / lesson:** Week 8 / Lesson 2 of 2

Starting point (2-2-1-3): *What did my last check (quiz, exit ticket, homework) show — and what am I changing today because of it?*

Task 8.1 uploads: 80%; test tables weak on boundary data. Today: arithmetic operators (incl. MOD, DIV, ^) and why order of statements matters.

1 OUTCOMES — What must students learn?

Desired learning outcome(s) — *trace to the curriculum / standard and the table of specifications (ToS)*

Students use arithmetic operators + - * / ^ MOD DIV in pseudocode and Python (**, //, %), apply operator precedence, and explain sequence. Syllabus 8.1.4(a), 8.1.4(f); AO1, AO2.

Success criteria — *what will successful learning look / sound like?*

I can evaluate expressions with MOD and DIV; I can convert pseudocode operators into Python; I can reorder jumbled statements into a correct sequence and explain why.

Key knowledge / skill / understanding:

DIV = whole-number quotient; MOD = remainder; precedence: brackets, ^, * / , + -; sequence = statements run in order.

Likely misconception / access or challenge need:

MOD is a percentage; / and DIV give the same result; order of lines doesn't matter.

2 PROOF — What will students do to show they have learned it?

Three activities, each with a DIFFERENT strategy, resource and evidence tool. Then tick the Coverage Check on page 2.

ACTIVITY 1 *student task + evidence of learning*

Strategy: Problem solving **Time:** 10 min

What students will do:

Calculate by hand (no calculators, as in Paper 2): 17 DIV 5, 17 MOD 5, 2^3, 10 - 4 / 2. Then convert 'seconds to minutes and seconds' using DIV and MOD.

Evidence tool: Mini-whiteboards

Assessment: ☑ Diagnostic ☐ Formative ☐ End

Grouping: ☑ Individual ☐ Pair ☐ Group

Resources:

Expression slide; whiteboards.

Support (low performers / SEN):

Number line and counters for DIV/MOD.

Stretch (fast learners / gifted):

Write an expression to check if a number is even.

ACTIVITY 2 *student task + evidence of learning*

Strategy: Jigsaw **Time:** 20 min

What students will do:

Expert groups master one program (time converter, change-giver, even/odd checker), write its pseudocode and Python, then teach it to a mixed group.

Evidence tool: Practical / performance task

Assessment: ☐ Diagnostic ☑ Formative ☐ End

Grouping: ☐ Individual ☐ Pair ☑ Group

Resources:

Coursebook (CUP 9781108915144) 8.4-8.5; expert cards; Python.

Support (low performers / SEN):

Expert card with partially written pseudocode.

Stretch (fast learners / gifted):

Change-giver for SAR notes 500/100/50/10/5/1 using DIV and MOD.

ACTIVITY 3 *student task + evidence of learning*

Strategy: Gamification **Time:** 7 min

What students will do:

Parsons puzzle race: reorder 8 jumbled lines of a 'shopping total' program into the correct sequence; justify one move.

Evidence tool: Exit ticket

Assessment: ☐ Diagnostic ☑ Formative ☐ End

Grouping: ☑ Individual ☐ Pair ☐ Group

Resources:

Parsons puzzle (paper strips or online).

Support (low performers / SEN):

Fewer lines (5).

Stretch (fast learners / gifted):

Find the line that can move without changing output.

3 PREPARE — What do I need to do so students can succeed?

Set students up to succeed:

☑ Hook / prior learning ☑ Explain or model ☑ Success criteria shared

☑ Key vocabulary / examples ☑ Real-life connection ☑ Clear instructions / resources

☑ Praise, recognition and reinforcement planned

My setup / modelling: Time: 6 min

Hook: 125 seconds on a stopwatch - how many minutes and seconds? Model DIV and MOD, then Python // and %. Show precedence with brackets. Explain jigsaw roles and timing.

What is new or innovative in this lesson? *(ETEC looks for this)*

Change-giver uses real Saudi riyal notes; jigsaw makes each student an expert who teaches a program.

Progress recorded / shared:

☑ Gradebook ☐ Parents ☑ Written feedback in notebook / portfolio

4 RESPOND — What will I do with what I see and hear?

If students struggle or finish early I will:

☑ Re-teach ☑ Regroup ☑ Give extra support

☑ Stretch ☑ Have students redo / improve

What I will watch for / likely responses:

Using / instead of DIV; ^ typed in Python (bitwise XOR, not power); precedence errors; statements in wrong order using a variable before input. Likely: '17 MOD 5 = 3.4'.

Close / exit check *(summative + time for learners' questions)* Time: 7 min

Exit ticket: evaluate 29 DIV 4 and 29 MOD 4; reorder 4 lines; one question for the teacher.

Homework / platform *(e.g. Madrasati)*

Coursebook 8.4 questions; write a program that converts minutes to hours and minutes using DIV and MOD.

5 REFLECT — After the lesson *Complete within a day. This is reflective-practice evidence and feeds your next Starting point.*

What worked — and how do I know?

Riyal change-giver was a hit and linked to real shopping. Hand calculation exposed MOD misconceptions early. 85% correct on exit ticket.

What will I change next time?

Warn about ^ in Python (bitwise XOR) before coding; it caused most wrong outputs. Give the Parsons strips on paper for faster sorting.

Students needing follow-up:

10BR3: 3 students still treat MOD as division; give extra counters practice. Inform parents of 2 missing uploads.

COVERAGE CHECK *Tick where each must-have happens (1 · 2 · 3 = activity, C = close, H = homework). An empty box is a gap to fix before teaching. Literacy and numeracy apply in every subject.*

Literacy	Numeracy	Digital	Digital research	Real-life ①	Real-life ②	Inquiry	Thinking
read + write · 2-1-1-6	numbers / data · 2-1-1-6	students use it · 2-1-1-4	trusted sources · 2-1-1-4	application · 2-1-1-5	application · 2-1-1-5	experiment / research · 2-1-1-7	problem-solving · 2-1-1-7
☐1 ☑2 ☐3 ☐C ☐H	☑1 ☐2 ☐3 ☑C ☐H	☐1 ☑2 ☐3 ☐C ☑H	☐1 ☐2 ☐3 ☐C ☑H	☐1 ☑2 ☐3 ☐C ☐H	☐1 ☐2 ☑3 ☐C ☐H	☑1 ☐2 ☐3 ☐C ☐H	☑1 ☐2 ☑3 ☐C ☐H

Create	Discussion	Pair / group	Learner choice	Self-assessment	Peer assessment	SEN / low	Gifted
something new · 2-1-1-7	expressing ideas · 2-1-1-8	cooperative work · 2-1-1-8	independent work · 3-2-1-6	2-2-1-1	2-2-1-1	specific support · 2-1-1-3	specific stretch · 2-1-1-3
☐1 ☑2 ☐3 ☐C ☐H	☐1 ☑2 ☐3 ☐C ☐H	☐1 ☑2 ☐3 ☐C ☐H	☐1 ☐2 ☐3 ☐C ☑H	☐1 ☐2 ☐3 ☑C ☐H	☐1 ☑2 ☐3 ☐C ☐H	☑1 ☑2 ☑3 ☐C ☐H	☑1 ☑2 ☑3 ☐C ☐H

ALWAYS VISIBLE IN DELIVERY: Safe, respectful climate • every learner drawn in • purposeful use of time • smooth transitions • learner voice • specific feedback and next steps • students act on feedback